To do the actual profiling, we use `cProfile` package, which is already included in the basic `Python` distro.

We won't explain in details how to use `cProfile` (lots of free material online covers this, starting from Python docs linked above), but to sum it up, `cProfile` allows running a method or function and records the per-call, total and cumulative time taken by every method involved.

```
 ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1    0.000    0.000    0.002    0.002 {built-in method builtins.exec}
      1    0.000    0.000    0.002    0.002 <string>:1(<module>)
      1    0.000    0.000    0.002    0.002 huffman_profile.py:24(run_test)
      1    0.000    0.000    0.002    0.002 huffman.py:116(create_encoding)
      1    0.000    0.000    0.001    0.001 huffman.py:92(_heap_to_tree)
     37    0.000    0.000    0.001    0.000 dway_heap.py:190(top)
     44    0.000    0.000    0.001    0.000 dway_heap.py:71(_push_down)
     95    0.000    0.000    0.001    0.000 dway_heap.py:141(_highest_priority_child_index)
780/452    0.000    0.000    0.000    0.000 {built-in method builtins.len}
      1    0.000    0.000    0.000    0.000 huffman_profile.py:11(read_text)
      1    0.000    0.000    0.000    0.000 huffman.py:72(_frequency_table_to_heap)
   37/1    0.000    0.000    0.000    0.000 huffman.py:49(tree_encoding)
      1    0.000    0.000    0.000    0.000 {built-in method io.open}
      1    0.000    0.000    0.000    0.000 dway_heap.py:18(__init__)
      1    0.000    0.000    0.000    0.000 dway_heap.py:169(_heapify)
    328    0.000    0.000    0.000    0.000 dway_heap.py:44(__len__)
     18    0.000    0.000    0.000    0.000 dway_heap.py:217(insert)
     45    0.000    0.000    0.000    0.000 dway_heap.py:166(first_leaf_index)
      1    0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
      1    0.000    0.000    0.000    0.000 huffman.py:67(_create_frequency_table)
```

Figure 2.18 Printing `Stats` after profiling Huffman encoding function.

Using `pStats.Stats`, we can retrieve and print (or process) those stats: the output of profiling looks something like what's shown in figure 2.18.